SECURITY AUDIT

Lista Dao

Surfin

FINAL REPORT

Aug 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – Surfin – Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/lista-new-contracts/tree/50c8c4078690a7766276e16ca8be4bdc47a39d60/src/surfin
Resolution 1	https://github.com/lista-dao/lista-new-contracts/tree/48a50827ed4833681045b50d98f5eb24ee897db0

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	4	2	1	1		
Low	16	8	4	4		
Informational	13	7		6		
Governance	1			1		
Total	34	17	5	12		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Privileged roles control external custody and principal-backed liquidity
Severity	**Governance**
Description	The system relies on trusted privileged roles and off-chain operations. The MANAGER controls the external Surfin wallet, InterestDistributor, fee receiver, floor rate, interest funding, and recall accounting. Assets deployed to the Surfin wallet leave contract custody and depend on external custody and recall operations before they can fund withdrawals. The MANAGER can also configure an arbitrary non-zero interest distributor and call fundInterest() using principal-backed adapter liquidity. DEFAULT_ADMIN_ROLE can transfer tokens through emergencyWithdraw() and can replace contract implementations through UUPS upgrades. The values supplied to settleRecall(), including the deployed book value and fee allocation within the recalled amount, are trusted inputs and are not independently verified on-chain. The BOT is also a liveness dependency for matured auto-renew positions. After maturity, the user cannot early-redeem, request a maturity withdrawal while auto-renew remains enabled, or disable auto-renew after the lock deadline. Only the BOT can progress the position through renewPosition(), which also requires an enabled destination cohort.
Recommendations	Protect privileged roles with appropriately secured multisignatures and timelocks. Monitor BOT renewal operations and maintain a replacement procedure. Consider allowing users to withdraw after a grace period if the BOT does not renew their matured position.
Comments / Resolution	Acknowledged.

FlexEarnPool

The FlexEarnPool contract is the flexible, no-maturity savings product built on the abstract CreditFundBase contract. CreditFundBase provides the shared FIFO withdrawal queue, batch funding and confirmation, claim logic, withdrawal accounting, limits, pauses, roles, and upgrade controls. FlexEarnPool adds deposits, flexible withdrawal requests, cancellation, and LP accounting. Deposits move assets directly to SurfinAdapter and create an equal non-transferable LP balance. requestWithdraw burns LP and queues the same principal amount, while cancelWithdraw restores LP if the request is still unconfirmed. The adapter funds the queue through finishWithdraw, after which the user or BOT can claim, with payment always sent to the user.

Privileged Functions

- finishWithdraw()
- setMinDeposit()
- setMinWithdraw()
- setDailyLimit()
- setDepositPaused()
- setAdapter()
- pause()
- unpause()
- emergencyWithdraw()

Invariants

- totalPrincipal() == totalSupply == sum of all user LP balances.
- totalPendingWithdraw equals the sum of all outstanding withdrawal requests.
- Requesting a withdrawal burns the same amount of LP cancelling an unconfirmed request restores exactly that amount.
- Batches are confirmed sequentially, and a request cannot be claimed before its batch is fully funded and confirmed.
- Assuming the configured asset transfers the exact requested amount, and excluding unsolicited transfers and emergency withdrawals, every confirmed withdrawal is fully backed by assets held in the pool and cannot be claimed twice.

Issue_02	Pausing blocks funded claims and withdrawal cancellations
Severity	Medium
Description	The pause mechanism is applied to functions that release or restore user assets, causing existing withdrawal and reward operations to become inaccessible. **Principal withdrawals:** claimWithdraw() is implemented in CreditFundBase and inherited by both FlexEarnPool and LockedEarnPool. After the adapter funds a withdrawal batch, the assets are already transferred into the pool and the requests are confirmed. However, if the pool is paused before a user claims, claimWithdraw() reverts because it uses whenNotPaused. The funded assets remain inside the pool until the MANAGER unpauses it. finishWithdraw() remains callable while the pool is paused. Therefore, the adapter can continue transferring assets into the pool and confirming withdrawals even though users cannot claim them. **Flex withdrawal cancellation:** When a Flex user requests a withdrawal, their LP is burned immediately. Before confirmation, cancelWithdraw() is the only way to reverse the request and restore that LP. Because cancellation is also protected by whenNotPaused, a paused user has neither their LP nor a claimable withdrawal until the pool is unpaused or the request is later funded. **Interest rewards:** InterestDistributor.claim() also uses whenNotPaused. Pausing the distributor therefore blocks valid reward claims even when the reward tokens are already funded and the Merkle root has been accepted.
Recommendations	Allow users to claim funded withdrawals, claim rewards, and cancel unconfirmed Flex withdrawals while the contracts are paused. Keep deposits and other new operations blocked.
Comments / Resolution	Acknowledged. Full pause intentionally blocks claims and cancellations, the risk is accepted.

Issue_03	Mutable withdrawal batches can delay FIFO settlement
Severity	**Low**
Description	The BOT determines a withdrawal batch's funding amount off-chain before submitting the settlement transaction. However, the batch remains mutable until it is confirmed, and finishWithdraw() receives only a funding amount without identifying the batch or the total observed by the BOT. New requests submitted on the same day can increase the open batch total. For example: - Alice requests 100 USDT, and the BOT prepares a transaction to fund 100 USDT. - Before the BOT transaction executes, Bob requests 10 USDT and joins the same batch. - The batch total becomes 110 USDT. - The BOT's 100 USDT is accepted as partial funding, but the batch cannot be confirmed until the remaining 10 USDT is provided. The opposite race can occur in FlexEarnPool. If a user cancels a request before the BOT transaction executes, the batch total and pending withdrawal amount decrease. A transaction prepared using the previous larger amount may then revert because it exceeds the remaining pending withdrawals. A similar race can affect settleRecall() if lockedCoverAmount is sized from totalPendingWithdraw. If a confirmed Locked withdrawal is claimed before execution, totalPendingWithdraw decreases and the stale cover amount can cause the complete recall transaction to revert. The transaction must then be resized and resubmitted. Because batches are confirmed in FIFO order, delaying the first unconfirmed batch also delays every later batch, partial funding remains available for a later top-up, while an excessive funding transaction reverts completely.

Recommendations	Fund and close the current batch in one transaction using its on-chain total. Once settlement begins, prevent new requests or cancellations from changing that batch and place new requests into the next batch.
Comments / Resolution	Acknowledged. The withdrawal batch remains mutable, so stale settlement sizing can still delay FIFO processing. The same request/cancellation race can invalidate max-sized interest-funding and deployment transactions. The settleRecall() claim race is mitigated by tracking only the unconfirmed obligation.

Issue_04	Incompatible withdrawal limits can prevent users from exiting FlexEarnPool
Severity	Low
Description	minWithdraw and dailyLimit can be configured independently without checking that their values are compatible. If dailyLimit is lower than minWithdraw, a user whose LP balance is greater than dailyLimit may have no valid withdrawal amount. For example: - minWithdraw is 500 USDT. - dailyLimit is 100 USDT. - Alice owns 1,000 USDT of LP. A withdrawal of 500 USDT satisfies minWithdraw but exceeds dailyLimit. A withdrawal of 100 USDT satisfies dailyLimit but is below minWithdraw and does not drain Alice's full balance. Withdrawing the complete 1,000 USDT balance also exceeds dailyLimit. Alice therefore cannot submit any withdrawal until the configuration is changed.
Recommendations	When both limits are enabled, require dailyLimit to be greater than or equal to minWithdraw.
Comments / Resolution	Fixed. Both setters now enforce minWithdraw <= dailyLimit whenever both limits are enabled.

Issue_05	Cancellation can leave a queued withdrawal below minWithdraw
Severity	**Low**
Description	requestWithdraw() permits an amount below minWithdraw when that amount equals the user's entire remaining LP balance. However, the check considers only the user's current LP balance and ignores other unconfirmed requests that may later be cancelled. **For example**, Alice owns 100 USDT of LP while minWithdraw is 50 USDT: - Alice requests 90 USDT, leaving a 10 USDT LP balance. - Alice requests the remaining 10 USDT. This is accepted because it drains her current balance. - Alice cancels the 90 USDT request and receives 90 USDT of LP back. Alice now owns 90 USDT of LP while her 10 USDT withdrawal remains queued below minWithdraw. The configured minimum can therefore be bypassed without permanently exiting the position, allowing dust requests to enter the settlement queue. This also allows dust withdrawals below the configured minimum to enter and modify an open withdrawal batch.
Recommendations	Permit a below-minimum request only when the user will have no remaining LP and no other cancellable withdrawal requests.
Comments / Resolution	Fixed. Below-minimum withdrawals now require a full exit with no cancellable requests.

Issue_06	totalPendingWithdraw can allow excess funding into the pool
Severity	**Low**
Description	totalPendingWithdraw includes every withdrawal that has not yet been claimed, cancelled, or reinvested. Therefore, it includes both unfunded requests and confirmed requests whose assets are already held by the pool. finishWithdraw() only verifies that withdrawQuota does not exceed totalPendingWithdraw. Because confirmed requests remain included, they can provide accounting room for additional funding that does not cover any unfunded request. For example: - Alice has a confirmed but unclaimed withdrawal of 500 USDT. - Alice opens another 500 USDT request. - The BOT prepares a transaction to fund the new request. - Alice cancels the new request before the BOT transaction executes. - totalPendingWithdraw returns to 500 USDT because Alice's confirmed request remains. - The BOT's stale 500 USDT funding transaction still passes the quota check. - The cancelled batch contains zero withdrawals, so the new 500 USDT remains as unused withdrawQuota. - After Alice claims her original request, the pool still holds 500 USDT with no current withdrawal liability. The unused quota can fund future Flex withdrawals. Until then, the assets are unavailable to the adapter and cannot fund Locked withdrawals or be deployed to Surfin.
Recommendations	Track funded and unfunded withdrawal liabilities separately. Limit positive funding to the current unfunded amount and bind settlement transactions to the expected batch ID and batch total.
Comments / Resolution	Fixed.

Issue_07	Stale adminTopUp can leave excess withdrawal quota in the pool
Severity	**Low**
Description	adminTopUp() allows the DEFAULT_ADMIN to inject funds and confirm queued batches during fund impairment. Unlike finishWithdraw(), which requires withdrawQuota <= _unconfirmedObligation() and reverts on excess funding, adminTopUp() deliberately bypasses this bound so that batches can still confirm when the adapter cannot pay. However, the open batch remains mutable until it is confirmed. A cancellation that executes between the admin sizing the top-up and the transaction landing shrinks the batch total, and bypass withdrawQuota <= _unconfirmedObligation() invariant, leaving trapped funds into the pool. Consider the following: - The open Flex batch totals 100 USDT and holds 70 USDT of withdrawQuota from partial funding. The remaining shortfall is 30 USDT. - The admin prepares adminTopUp(30) to complete the batch. - Before the transaction executes, Alice cancels her 30 USDT request. The batch total falls to 70 USDT and the unconfirmed obligation falls to 70 USDT. - adminTopUp(30) executes without any quota check. withdrawQuota becomes 100 USDT and _confirmBatches() confirms the 70 USDT batch. - The final state is withdrawQuota = 30 against an unconfirmed obligation of 0. The invariant withdrawQuota <= _unconfirmedObligation(), preserved by every other entry point, is broken. The 30 USDT is now trapped as unassigned quota inside the pool. It is not returned to the admin, and it is not available to the adapter for funding Locked withdrawals or deployment to Surfin.
Recommendations	Sweep the excess at the end of adminTopUp(): after _confirmBatches(), if withdrawQuota exceeds _unconfirmedObligation(), return the

	difference to msg.sender (or the adapter) and reduce the quota accordingly.
Comments / Resolution	Acknowledged.

Issue_08	A BOT claim can retarget a pending Flex withdrawal cancellation
Severity	**Low**
Description	Withdrawal requests are referenced by their current array index. When a confirmed request is claimed, _consumeConfirmedWithdraw() removes it using swap-and-pop, which can move another request into the same index. **For example:** - Alice submits cancelWithdraw(0) for unconfirmed request A. - Before Alice's transaction executes, request A is funded and confirmed. - The BOT claims request A first. - The claim removes request A and moves a later unconfirmed request B into index 0. - Alice's pending cancelWithdraw(0) then cancels request B instead. Alice unintentionally cancels a different withdrawal and loses its existing position in the withdrawal queue.
Recommendations	require cancelWithdraw() to verify the expected amount and batch ID before cancelling the indexed request.
Comments / Resolution	Partially Fixed. expectedAmount prevents mismatched amounts, but same-sized requests can still be retargeted because the batch ID is not verified.

Issue_09	emergencyWithdraw(asset) can make totalConfirmedUnclaimed return funds when they are not
Severity	Low
Description	totalConfirmedUnclaimed variable is newly introduced and its the confirmed withdrawal funds already funded and sitting inside the pool. The adapter trusts this value. If a batch is confirmed then the adapter no longer reserves funds for it because the pool already has the funds. But emergencyWithdraw() can transfer the pool's asset out without updating totalConfirmedUnclaimed or withdrawQuota. So an admin could rescue USDT from the pool after withdrawals are confirmed. The accounting would still say those claims are funded but the actual token balance may be gone. Users' claims can then fail, while the adapter still thinks everything is fine and may allow deploys or interest funding based on stale accounting.
Recommendations	Prevent rescuing the asset below withdrawQuota + totalConfirmedUnclaimed else consider acknowledging the behaviour.
Comments / Resolution	Acknowledged.

Issue_10	Cancelled Flex withdrawals can leave funds inside the Flex pool
Severity	**Informational**
Description	The BOT can send part of a withdrawal amount from the adapter to FlexEarnPool. Before the batch is fully funded, the user can cancel their request and receive their LP back. However, the money already sent remains inside FlexEarnPool. Future Flex withdrawals can use this money. Until then, the adapter cannot use it to fund Locked withdrawals, which may delay those users when liquidity is limited.
Recommendations	After a cancellation, return any unused withdrawal funds to the adapter.
Comments / Resolution	Fixed. Cancellation now returns any excess withdrawal funds to the adapter.

Issue_11	cancelSupported() could be a footgun if cancelSupported() Is toggled in between
Severity	**Informational**
Description	The dust exit protection relies on _hasCancellableRequest() but _hasCancellableRequest() queries _cancelSupported() whether this pool supports cancellation. The problem is that _cancelSupported() is declared as a virtual function in the base contract so any future pool implementation or an external integration can override it and change the returned value from false to true , consider the following → 1. Locked currently does not support cancellation so _cancelSupported() returns false. 2. A user has a large unconfirmed locked withdrawal. 3. The user makes a tiny below minimum dust exit and _hasCancellableRequest() ignores the large request because cancellation is unsupported. 4. Later, Locked is upgraded and overrides _cancelSupported() to return true. 5. That old large request may now be cancellable. 6. The user cancels it while the tiny dust withdrawal remains queued.
Recommendations	Consider acknowledging the behaviour.
Comments / Resolution	Acknowledged.

LockedEarnPool

The LockedEarnPool contract is the fixed-term savings product built on the abstract CreditFundBase contract. Deposits create locked positions inside configured cohorts and transfer the assets directly to SurfinAdapter. Each cohort defines its deposit deadline and maturity. Users may redeem before maturity, potentially paying an early-redemption penalty, or withdraw their full principal after maturity. The BOT can batch matured withdrawals and renew auto-renew positions into another enabled cohort. Users can also reinvest a confirmed withdrawal into a new locked position. CreditFundBase provides the shared FIFO withdrawal queue, funding, claims, accounting, limits, pauses, roles, and upgrade controls.

Privileged Functions

- setCohort()
- setPenaltyRate()
- setMinDeposit()
- setMinWithdraw()
- setDailyLimit()
- setDepositPaused()
- setAdapter()
- pause()
- unpause()
- emergencyWithdraw()
- finishWithdraw()
- claimWithdraw()
- batchRequestMaturityWithdraw()
- renewPosition()

Invariants

- totalPrincipal() == totalPrincipalAmount == the sum of principal in all open positions.
- Depositing `x` creates an open position with `x` principal, increases total principal by `x`, and transfers `x` assets to the adapter.
- Early redemption decreases the position and total principal by the gross redeemed amount, while queuing the payout after any applicable penalty.
- The early-redemption penalty never exceeds 10% and applies only during the first 30 days of a position.
- A maturity withdrawal closes a matured, non-auto-renewing position and queues its full principal.
- Renewal closes the old position and creates a new position with the same principal, leaving total principal unchanged.

Issue_12	BOT can retroactively change the maturity of existing locked positions
Severity	Medium
Description	Locked positions store only a cohortId. Their maturity is not stored inside the position and is instead read from the current configuration of that cohort. The BOT-controlled setCohort() function can overwrite a cohort's term, deposit deadline, maturity, and enabled status even after users have deposited into it. Each update verifies that the new maturity matches the newly supplied term and deposit deadline. However, both values are also selected by the BOT. The checks therefore validate the new schedule without preserving the schedule that users originally accepted. The same mutability affects pending deposits because deposit() accepts only a cohort ID and does not bind the transaction to the term, deposit deadline, or maturity observed by the user. A cohort update executed first can therefore make the pending deposit enter under different terms. For example, Alice deposits into a cohort scheduled to mature in three months. The BOT can later overwrite the same cohort ID with a ten-year term and matching maturity. Alice's position immediately follows the new maturity date. She can no longer withdraw normally after the original three-month term and must either wait for the extended maturity or redeem early and forfeit interest. The mutable maturity also affects early-redemption eligibility, maturity withdrawals, auto-renew changes, and position renewals. This gives the BOT authority to retroactively change the lifecycle of existing positions without user consent.
Recommendations	Require deposit() to bind the expected term, deposit deadline, and

	maturity, or their configuration hash, and revert if the live cohort configuration differs.
Comments / Resolution	Partially Fixed. BOT can no longer modify existing cohort dates, but MANAGER can repeatedly shift maturity by 31 days and deposits remain unbound to the expected cohort configuration.

Issue_13	renewPosition can renew a position into a stale cohort
Severity	**Low**
Description	Normal deposits and reinvest check whether the selected cohort is still accepting deposits. renewPosition only checks that the new cohort is enabled and does not apply the same deposit deadline and maturity checks. Consider the following → a. Cohort 7 is enabled with a deposit deadline of August 1 and a maturity date of October 1. b. Alice's old position matures on August 10 with autoRenew = true. c. On August 11 the bot calls renewPosition(positionId, 7). d. A fresh deposit or reinvest into cohort 7 would revert because the August 1 deadline has passed. e. renewPosition succeeds because cohort 7 is still marked enabled. f. Alice is placed into a cohort that is no longer meant to accept principal.
Recommendations	Apply the same cohort checks used by deposit and reinvest
Comments / Resolution	Fixed. renewPosition() now enforces the target cohort's deposit deadline like deposits and reinvestments.

Issue_14	Users can lose their auto-renew opt-out while renewal bypasses pause controls
Severity	Low
Description	reinvest() opens a new locked position and is protected by both whenNotPaused and whenDepositNotPaused. renewPosition() also opens a new locked position but has neither modifier. The pause behavior is also asymmetric. toggleAutoRenew() is blocked while the contract is paused, but the BOT can still renew the position. If a pause overlaps the user's final opt-out window, the user may lose the opportunity to disable auto-renew for that renewal cycle For example: - The contract is paused 60 days before maturity, preventing Alice from disabling auto-renew. - The contract is unpaused 20 days before maturity. - Alice is now inside the 32-day lock window and still cannot disable auto-renew. - At maturity, the BOT can renew her position into another term. The renewed position resets depositTime, placing the principal inside a new early-redemption penalty window.
Recommendations	Add both whenNotPaused and whenDepositNotPaused to the renewal path else consider acknowledging the issue.
Comments / Resolution	Partially Fixed. Renewal now respects both pause controls, but a pause can still consume the user's final opt-out window before a later renewal.

Issue_15	Wind-down can strand matured auto-renew positions
Severity	**Low**
Description	The fix for the renewal pause-bypass added both whenNotPaused and whenDepositNotPaused to renewPosition(). The second modifier creates a deadlock that did not exist before the fix. During wind-down (depositPaused = true), a matured position with auto-renew enabled has no executable path: - renewPosition() reverts on whenDepositNotPaused, - requestMaturityWithdraw() and the BOT batch reject it because auto-renew is on, - toggleAutoRenew() is frozen forever once maturity has passed, and early redemption is both deposit-paused and matured. The wind-down mode is documented as keeping "normal & matured withdrawals and claims" open, but for this class of position it closes every exit. Consider a user who enabled auto-renew and did not toggle it off before T-32: if the protocol enters wind-down before the BOT renews at maturity, their principal is frozen for the entire wind-down.
Recommendations	Allow the OWNER to manage the matured autorenew positions in a wind-down state.
Comments / Resolution	Acknowledged.

Issue_16	Penalty-rate changes can reduce early-redemption payouts
Severity	**Low**

Description	During the first 30 days of a position, the early-redemption payout depends on the mutable global penaltyRate. Users can preview their payout, but requestEarlyRedeem() recalculates it when the transaction executes:
	
	`function requestEarlyRedeem(uint256 posId, uint256 amount) external {`
	`  uint256 payout = _earlyRedeemPayout(pos.depositTime, amount);`
	`  // ...`
	`  _enqueueWithdraw(msg.sender, payout, cohortId);`
	`}`
	
	The MANAGER can change the rate at any time, up to 10%:
	
	`function setPenaltyRate(uint256 newRate) external onlyRole(MANAGER) {`
	`  require(newRate <= MAX_PENALTY_RATE, "penalty rate too high");`
	`  penaltyRate = newRate;`
	`}`
	
	However, requestEarlyRedeem() does not let the user specify a minimum payout or transaction deadline.
	
	For example, Alice previews a 50,000 USDT redemption at the default 0.8% penalty and expects 49,600 USDT. If the MANAGER raises the penalty to 10% before her transaction executes, the transaction still succeeds and queues only 45,000 USDT. The full 50,000 USDT is removed from her position, and the resulting withdrawal request cannot be cancelled.
	
	The user therefore cannot enforce the minimum amount they agreed to receive when submitting the transaction.

Recommendations	Add minPayout and deadline parameters to requestEarlyRedeem(). Revert if the calculated payout is below minPayout or the transaction executes after the deadline.
Comments / Resolution	Fixed. requestEarlyRedeem() now enforces user-specified minPayout and deadline protections.

Issue_17	Mutable withdrawal indices can cause unintended reinvestments
Severity	**Low**
Description	reinvest() selects a confirmed withdrawal using its array index. When claimWithdraw() removes a request, the last request in the array is moved into the removed index. For example, Alice submits reinvest(0) for a confirmed 100 USDT request. If the BOT claims that request first, another confirmed request may move into index 0. Alice's pending transaction then succeeds and reinvests the moved request instead, potentially locking a different amount than she intended.
Recommendations	Assign each withdrawal request a permanent ID and use that ID for claims and reinvestments.
Comments / Resolution	Partially Fixed. Mutable indices can still retarget same-sized withdrawals.

Issue_18	Short-duration cohorts can make auto-renew impossible to disable
Severity	**Low**
Description	setCohort() permits terms as short as one day, but toggleAutoRenew() only works when maturity is more than 32 days away. Therefore, if a user deposits with auto-renew enabled into a cohort with 32 days or less remaining, they cannot disable it even immediately after depositing. After maturity, early redemption is unavailable and maturity withdrawal reverts because auto-renew remains enabled. The user then depends on the BOT to renew the position.
Recommendations	Reject cohorts whose term is not longer than the 32-day auto-renew lock window, or prevent users from enabling auto-renew when 32 days or less remain.
Comments / Resolution	Fixed. Cohort terms must now exceed the 32-day auto-renew lock window.

Issue_19	A stale position can revert the entire BOT maturity-withdrawal batch
Severity	**Low**

Description	batchRequestMaturityWithdraw() processes multiple positions in one transaction:
	
	`for (uint256 i = 0; i < users.length; i++) {`
	`  _requestMaturityWithdraw(users[i], posIds[i]);`
	`}`
	
	The complete transaction reverts if any position is invalid. A position can become stale after the BOT prepares its transaction for example, if its owner submits requestMaturityWithdraw() first.
	
	If Alice's stale position and Bob's valid position are included in the same BOT transaction, Alice's entry reverts the transaction and Bob's withdrawal is not queued. The BOT must remove the stale entry and submit the batch again, delaying every other user included in it.

Recommendations	Check each position before processing it and skip entries that are already closed, not matured, or set to auto-renew. Emit an event for every skipped entry so the BOT can investigate it.
Comments / Resolution	Fixed.

Issue_20	Early redemption penalty rounds down in the user's favor
Severity	**Informational**
Description	The early redemption penalty is rounded down → uint256 penalty = (amount * penaltyRate) / PRECISION; i.e. against the favour of the protocol , rounding direction should be ideally in favour of the protocol and against the user.
Recommendations	Consider rounding up the penalty calculation else consider acknowledging the issue.
Comments / Resolution	Fixed. The early-redemption penalty now rounds up in the protocol's favor.

Issue_21	A position can be reduced after the auto renew interest snapshot without invalidating its allocation
Severity	Informational
Description	The protocol relies on an offchain calculator and Merkle roots to allocate interest. If that calculator takes a final auto renew snapshot at T−32, a user can reduce the snapshotted position afterward through requestEarlyRedeem without automatically invalidating the earlier allocation onchain. Consider the following case → a. Alice has an auto renew position containing 1,000 USDT. b. At T−32 the off-chain calculator snapshots the position as autoRenew = true with 1,000 USDT and calculates the corresponding full term or renewal interest. c. After the snapshot Alice calls requestEarlyRedeem(999 USDT). d. The position stays open with 1 USDT and remains marked for auto renewal. e. If the calculator does not process the redemption event and rebuild the allocation, the Merkle root still grants Alice interest based on 1,000 USDT. f. Alice claims that allocation even though only 1 USDT remains in the position and is ultimately renewed.
Recommendations	Make sure the offchain calculator takes the scenario into account and acknowledge the issue.
Comments / Resolution	Acknowledged.

Issue_22	Unconfirmed early-redemption requests retain their penalty after maturity
Severity	Informational
Description	requestEarlyRedeem() calculates the user's payout when the request is submitted. If the request is made during the penalty window, the contract immediately deducts the penalty, removes the full redeemed principal from the position, and queues only the reduced payout. Locked withdrawal requests cannot be cancelled. If the request remains unconfirmed until after the cohort matures, neither batch confirmation nor claimWithdraw() recalculates its value. The user therefore receives the same reduced amount calculated at request time. For example, Alice early-redeems 50,000 USDT while the default 0.8% penalty applies. The contract closes her position and queues 49,600 USDT. If the request is not confirmed until after maturity, Alice still receives only 49,600 USDT, even though she did not receive early access to her funds. The user may pay an early-exit penalty without receiving early liquidity because withdrawal timing depends on the BOT-operated settlement process. The difference can reach 10% under the maximum permitted penalty rate.
Recommendations	Define and enforce the intended penalty policy. If an early-redemption request remains unconfirmed through maturity, update its payout to the full principal before funding and confirming the batch.
Comments / Resolution	Acknowledged.

Issue_23	previewEarlyRedeem can quote redemptions that would revert
Severity	Informational
Description	previewEarlyRedeem() calculates a payout without checking whether the position is open, whether the requested amount exceeds its principal, or whether the position has already matured. As a result, the function can return a payout for a closed position, an excessive amount, or a matured position for which requestEarlyRedeem() would revert. Applications relying on the preview may display a valid-looking quote for a transaction that cannot execute.
Recommendations	Apply the relevant position, amount, and maturity checks inside previewEarlyRedeem().
Comments / Resolution	Fixed. previewEarlyRedeem() now validates the position, amount, and maturity.

Issue_24	Auto-renew documentation states a 30-day lock while the contract enforces 32 days
Severity	Informational
Description	AUTO_RENEW_LOCK_WINDOW is set to 32 days, and toggleAutoRenew() enforces that value. However, the function documentation and revert message describe the restriction as T−30. A user relying on the documented deadline may attempt to change auto-renew between T−32 and T−30 and find that the option is already locked.
Recommendations	Update the documentation and revert message to state T−32. If T−30 is intended, change the enforced lock window to 30 days.
Comments / Resolution	Fixed.

Issue_25	Auto-renew allows deposit below minDeposit
Severity	Informational
Description	Fresh deposits and reinvestments must satisfy minDeposit. However, a user can partially early-redeem an existing auto-renew position until its remaining principal is below minDeposit. At maturity, renewPosition() can still close the old position and create a new position containing that below-minimum principal because renewal does not enforce minDeposit. Renewal moves existing principal rather than introducing new assets, so this does not directly create a solvency issue. However, it creates inconsistent minimum-position behavior between deposits, reinvestments, and auto-renewal and may preserve operationally undesirable dust positions.
Recommendations	Consider acknowledging this issue
Comments / Resolution	Acknowledged.

InterestDistributor

The InterestDistributor contract distributes Flex and Locked interest using cumulative Merkle proofs. The FUNDER transfers reward tokens into the contract, while the BOT stages a new Merkle root and activates it after a review period. Users or third parties may submit valid claims, but rewards are always transferred to the account encoded in the proof. Each root contains cumulative reward amounts, so users receive only the difference between their new allocation and the amount already claimed. The manager can revoke pending roots, change the review period, and perform emergency withdrawals.

Privileged Functions

- notifyReward()
- setPendingMerkleRoot()
- acceptMerkleRoot()
- revokePendingMerkleRoot()
- changeWaitingPeriod()
- emergencyWithdraw()
- pause()
- unpause()

Invariants

- claimed[account] only increases, and each claim transfers exactly the difference between the new cumulative allocation and the amount already claimed.
- The same cumulative reward allocation cannot be claimed twice.
- Only one root can be pending, and it cannot be accepted before a waiting period of at least six hours.

Issue_26	Merkle-root waiting period is mutable and has no upper bound
Severity	**Informational**
Description	changeWaitingPeriod() can modify the delay after a root is staged and allows any value above six hours. This can shorten the expected review period or accidentally delay root activation for an excessive time. The source comment describes a one-day default waiting period, while initialize() sets it to six hours.
Recommendations	Store each root's activation time when it is staged and set a reasonable maximum waiting period. Update the source comment to match the six-hour default.
Comments / Resolution	Fixed.

Issue_27	One stale claim can revert an entire batchClaim transaction
Severity	Informational
Description	batchClaim() processes every claim atomically. Each entry calls claim(), which reverts when the supplied cumulative amount is not greater than the amount already claimed by that account. Because claims are permissionless, any party can execute one valid claim before a prepared batchClaim() transaction. When the batch later processes that same entry, it reverts and rolls back every other claim in the batch. The claimed user still receives their correct reward, but the remaining users are delayed and the batch submitter must rebuild and resubmit the transaction.
Recommendations	Skip entries whose cumulative amount has already been claimed and emit an event for each skipped account.
Comments / Resolution	Fixed.

SurfinAdapter

The SurfinAdapter contract is the shared custody and liquidity-routing contract for FlexEarnPool and LockedEarnPool. Both pools transfer user deposits directly into the adapter. The manager can deploy available assets to the external Surfin wallet through deployToSurfin and record returned assets through settleRecall. The BOT funds the Flex and Locked withdrawal queues and transfers accrued fees to the configured fee receiver. The manager can also fund InterestDistributor and configure the floor rate and destination addresses. The adapter protects accrued fees and a calculated hard floor during normal deployment and withdrawal-funding operations.

Privileged Functions

- deployToSurfin()
- settleRecall()
- finishFlexWithdraw()
- finishLockedWithdraw()
- fundInterest()
- claimFee()
- pause()
- unpause()
- setFloorRate()
- setSurfinWallet()
- setInterestDistributor()
- setFeeReceiver()
- emergencyWithdraw()

Invariants

- hardFloor() equals floorRate multiplied by the total principal and pending withdrawals of both pools, and floorRate cannot exceed 100%.
- maxDeployToSurfin() and available withdrawal liquidity equal the adapter balance remaining after reserving accrued fees and the hard floor.
- A successful fee claim reduces the adapter balance and accruedFee by the same amount and sends the assets only to feeReceiver.
- freeIdle() == max(idleBalance - accruedFee, 0).

Issue_28	Funded but unclaimed withdrawals are still counted in the adapter's floor base
Severity	Medium
Description	The adapter calculates its liquidity floor using live principal plus totalPendingWithdraw. The problem is that a withdrawal stays in totalPendingWithdraw until the user actually claims it, even after the adapter has already sent the withdrawal funds to the pool. Consider the following scenario → a. Alice deposits 100 USDT and Bob deposits 100 USDT. b. The adapter holds 200 USDT and the floor base is 200 USDT. c. Bob requests a withdrawal of 100 USDT. d. Bob's principal is burned, so live principal falls to 100 USDT, while totalPendingWithdraw becomes 100 USDT. e. The floor base remains 200 USDT. This is fine at this stage because Bob's 100 USDT is still in the adapter. f. The adapter transfers 100 USDT to the pool to fund Bob's request. g. The adapter now holds only the 100 USDT backing Alice, so its logical floor base should be 100 USDT. h. Bob does not claim. Because totalPendingWithdraw is reduced only on claim, it remains 100 USDT. i. The adapter still calculates a floor base of 100 + 100 = 200 USDT. j. At a 3% floor rate, it protects 6 USDT instead of 3 USDT.
Recommendations	Update the floor base calculation to exclude withdrawals that have already been funded.
Comments / Resolution	Fixed. The floor base now excludes funded withdrawals whose assets have left the adapter.

Issue_29	Partially funded Locked withdrawal batches can become blocked by the hard floor
Severity	Medium
Description	SurfinAdapter calculates its hard floor from both pools' principal and totalPendingWithdraw. However, transferring withdrawal funds from the adapter into a pool does not reduce totalPendingWithdraw. If the adapter can fund only part of a Locked withdrawal batch, the transferred funds remain as withdrawQuota, but the batch is not confirmed and cannot be claimed. The unchanged hard floor may then prevent the adapter from sending the remaining amount. **For example:** - The adapter holds 100 USDT. - The floor base consists of 2 USDT of live principal and a 98 USDT pending Locked batch. - At a 3% floor rate, the hard floor is 3 USDT. - The adapter can transfer only 97 USDT without crossing the floor. - The batch requires 98 USDT, so the transferred 97 USDT is insufficient to confirm it. - The adapter now holds 3 USDT, while its available withdrawal liquidity is zero because the unchanged floor remains 3 USDT. The 97 USDT inside LockedEarnPool cannot be claimed because the batch remains unconfirmed. FIFO ordering also prevents later batches from being confirmed. This condition continues until additional liquidity arrives, the floor rate is reduced, or another privileged action releases sufficient liquidity.
Recommendations	Calculate the hard floor using only liabilities whose funds remain inside the adapter. Amounts already transferred into a withdrawal pool should no longer increase the adapter's floor requirement.
Comments / Resolution	Fixed. Partial funding now reduces the floor base, allowing the remaining Locked withdrawal to be funded.

Issue_30	fundInterest can pay interest using fresh user principal
Severity	**Low**
Description	fundInterest is limited by freeIdle(), but freeIdle() only removes accrued fees from the adapter's token balance. It does not distinguish realized yield from user principal. *require(amount <= freeIdle(), "insufficient idle");* *IERC20(asset).safeIncreaseAllowance(interestDistributor, amount);* *IInterestDistributor(interestDistributor).notifyReward(amount);* Consider the following → a. Alice deposits 100,000 USDT. b. The pool records a 100,000 USDT principal liability to Alice. c. The adapter receives the 100,000 USDT but it has not deployed anything to Surfin and no yield has been realized. d. Because the balance is idle and is not booked as accruedFee, freeIdle() returns 100,000 USDT. e. The manager calls fundInterest(100_000e6). f. The entire deposit is transferred to the interest distributor. g. Alice's 100,000 USDT principal still exists in the pool's accounting, while the adapter has no liquidity left to back it. h. If the distributor pays that money out as interest , the protocol has paid interest from principal and now needs future yield, a recall, or new deposits to cover the missing assets. Because interest funding and withdrawal funding occur through separate privileged transactions, their ordering also affects withdrawal availability. If the MANAGER calls fundInterest() before the BOT funds a pending queue, the adapter balance and available withdrawal liquidity may fall below the required amount. The withdrawal transaction then reverts and users must wait for additional liquidity or a later recall.
Recommendations	Fund interest only from separately accounted realized yield. At minimum, reserve the liquidity required for current unfunded withdrawal queues before allowing assets to be transferred to InterestDistributor.

Comments / Resolution	Partially Fixed. Unfunded queues are reserved, but fundInterest() can still use fresh principal. Because withdrawals cannot pierce the hard floor, a max-sized interest funding can leave the first queued batch partially funded and blocked by the hard-floor amount.

Issue_31	deployToSurfin can deploy liquidity needed for pending withdrawals
Severity	**Low**
Description	maxDeployToSurfin() reserves accrued fees and the hard floor but does not reserve assets required by pending withdrawal queues. Requesting a withdrawal moves accounting from principal into totalPendingWithdraw, leaving the floor base and deployment limit unchanged. For example: - Alice deposits 100,000 USDT. - Alice requests a 50,000 USDT withdrawal. - The adapter holds enough liquidity to fund the complete request. - maxDeployToSurfin() still allows the MANAGER to deploy 97,000 USDT. - After deployment, only the 3,000 USDT hard floor remains and available withdrawal liquidity becomes zero. - The BOT can no longer fund Alice's 50,000 USDT request until assets are recalled or new liquidity arrives. The protocol expects the MANAGER to maintain an off-chain liquidity buffer, but the on-chain deployment limit does not enforce that pending withdrawals remain fundable.
Recommendations	Reserve the liquidity needed for pending withdrawal queues when calculating the deployment limit. Only assets remaining after the current queue obligations and required reserve should be deployable to Surfin
Comments / Resolution	Fixed.

Issue_32	Adapter pause blocks recall settlement while selected outflows remain enabled
Severity	Informational
Description	SurfinAdapter applies whenNotPaused to deployToSurfin() and settleRecall(), but not to fundInterest() or claimFee(). As a result, pausing the adapter blocks settleRecall(), which returns assets and can fund the Locked withdrawal queue, while interest and fee transfers remain callable. fundInterest() remains executable when the adapter is paused if InterestDistributor itself is not paused. The adapter pause therefore does not provide a consistent emergency-containment boundary.
Recommendations	Define the intended scope of the adapter pause explicitly.
Comments / Resolution	Acknowledged.

Issue_33	Emergency withdrawal events omit the receiver address
Severity	Informational
Description	The emergency withdrawal functions in SurfinAdapter and CreditFundBase transfer tokens to a caller-supplied receiver. However, their protocol events emit only the token address and amount. Monitoring systems that rely on the protocol's emergency-withdrawal events therefore cannot directly determine which address received the funds. The token's Transfer event may still expose the destination, so the impact is limited to protocol-event observability.
Recommendations	Include the receiver as an indexed parameter in each emergency-withdrawal event.
Comments / Resolution	Fixed. Emergency-withdrawal events now include the indexed receiver.

Issue_34	Credit losses are not reflected pro rata in user principal
Severity	**Information**
Description	The pools record principal liabilities that never adjust to the value of the assets backing them. FlexEarnPool mints LP strictly 1:1 and pays withdrawals at face value; LockedEarnPool positions store their principal and mature at face value. On the adapter, deployedToSurfin is a pure book value: settleRecall() resets it absolutely and bidirectionally, so a Surfin credit loss is recognized on-chain as a lower book — but nothing propagates that write-down to the pools. There is no exchange rate, no haircut factor, and no pro-rata write-down anywhere in the system. The real consequence is borne entirely by the last redeemers: once liquidity is exhausted, their requests sit unconfirmed in the queue. They cannot withdraw at all unless the protocol itself refunds the shortfall via adminTopUp() or a future recall.
Recommendations	Consider taking this into account off-chain.
Comments / Resolution	Acknowledged.